Cover sheet for submission of work for assessment

SWINBURNE UNIVERSITY OF TECHNOLOGY

UNIT DETAILS

Unit name	Introduction to Artificial Intelligence			Class day/time	Saturday Morning	Office use only
Unit code	COS30019	Assignment no.	2A	Due date	01/03/2026	
Name of lecturer/teacher	Dr Quang Chiem					
Tutor/marker's name						Faculty or school date stamp

STUDENT(S)

	Family Name(s)	Given Name(s)	Student ID Number(s)
(1)	Nguyen	Xuan Nang Mai	105549980
(2)	Luong	My An	105195040
(3)	Ngo	Quynh Nhu	105312140
(4)			
(5)			
(6)			

DECLARATION AND STATEMENT OF AUTHORSHIP

1. I/we have not impersonated, or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.

I/we understand that:

6. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

(1)		(4)	
(2)		(5)	
(3)		(6)	

Further information relating to the penalties for plagiarism, which range from a formal caution to expulsion from the University is contained on the Current Students website at **www.swin.edu.au/student/**

Copies of this form can be downloaded from the Student Forms web page at **www.swinburne.edu.au/studentforms/**

Table of Contents

Table of Contents 1
1. Instructions 3
1.1 Running an Individual Search 3
1.2 System Architecture Overview 3
2. Introduction 3
2.1 Problem Overview 3
2.2 Breadth First Search (BFS) 3
2.3 Depth First Search (DFS) 4
2.4 Uniform cost search (UCS) - CUS1 4
2.5 Greedy Best-First Search (GBFS) 5
2.6 A* Search (AS) 5
2.7 Step-Optimal A* (CUS2) 6
3. Features/Bugs/Missing 6
3.1 Modular Project Architecture 6
3.2 Implemented Features 7
3.3 Command Lines Execution 7
3.4 Bugs or Issues 7
4. Testing 7
Test Case 1: Simple Baseline Path Assessment 8
Test Case 2: Cycle Management and Exit Detection 8
Test Case 3: Directed Edge Constraint Verification 8
Test Case 4: Node-ID Tie Breaking Consistency 8
Test Case 5: Chronological Tie Resolution across Branches 8
Test Case 6: Deceptive Heuristic Path 8
Test Case 7: Least Cost versus Least Moves 9
Test Case 8: Same Edge Costs across All Paths 9
Test Case 9: Multiple Destinations with Unreachable Goal 9
Test Case 10: Multiple Destinations with Different Costs and Depths 9
Test Case 11: Isolated Destination Node (No Valid Path) 9
Test Case 12: Origin as Destination 9
Test Case 13: Dead-End and Backtracking 9
5. Insights 9
5.1 Data Collection and Evaluation Method 9
5.2 Efficiency: Nodes Created 9
5.3 Optimality: Path Cost 10
5.4 Solution Length: Path Length 10
5.5 Overall Learning 10
6. Research 11
6.1 Heuristic Admissibility and Optimality in A* 11
6.2 Step-Based Heuristic Design for CUS2 11
6.3 Deterministic Tie-Breaking and Reproducibility 11
6.4 Summary of Research Contribution 11

7. Conclusion..... 12
8. Acknowledgements/Resources.....12
References..... 12

1. Instructions

1.1 Running an Individual Search

The program is executed through a command-line interface. To run a single search instance, users navigate to the project root directory and execute: python search.py <filename> <method>. Here, <filename> refers to a graph file located in the testcases directory, and <method> denotes one of the supported algorithms: BFS, DFS, GBFS, CUS1, AS, or CUS2.

The system parses the input file, constructs the graph structure, invokes the selected algorithm, and prints three lines of output: the filename and method used, the goal reached together with the number of nodes created, and the solution path. If no valid path exists, the program reports None while still displaying the number of nodes generated.

1.2 System Architecture Overview

The implementation follows a modular architecture. The parser module reads and validates the structured input file. The graph module constructs adjacency structures and manages node coordinates. Each search algorithm is implemented independently in the algorithms directory to ensure separation of concerns. Supporting functionality is centralized in the utils module, which includes heuristic computations and a custom priority queue implementation. The priority queue explicitly handles deterministic tie-breaking, ensuring stable and reproducible expansion order across repeated runs.

2. Introduction

2.1 Problem Overview

The Route Finding Problem is a provide classical Artificial Intelligence search problem where the goal is to find a valid path between a specification origin node and one or more destinations nodes within a graph. Besides achieving a goal, the solution must frequently meet optimality requirements, including the cost of the total path or steps.

The environment in this assignment is represented as a directed, weighted graph with each node denoting a location and each edge denotes a directed relationship with a traversal cost associated with it. Due to the directional nature of edges, movement has to be stipulated within the connections.The search process is a tree building operation starting at the start, adding successor nodes, but recording parent references and path data like cost or depth. Upon reaching a destination, the path of the solution is rebuilt by driving parent links back to the root.

This problem is appropriate in the evaluations and comparison of various uninformed and informed search strategies because of the presence of cycles, multiple objectives, different edge costs, and heuristic directions.

2.2 Breadth First Search (BFS)

Breadth-First Search (BFS) is an uninformed method of search strategy that investigates the graph level after level, extending all the nodes at the present depth prior to proceeding to the subsequent level. It serves a FIFO queue to have the nodes that are generated earlier being expanded. Due to this level-order property, BFS ensures a minimum-step solution in graphs with actions equal in significance. In this assignment, BFS builds a search tree starting with the initial

node. Every node has its state, reference to its parent and depth. Neighbors are extended in completion of node ID order to meet deterministic tie-breaking. Upon a destination being reached, parent pointers are used to rebuild the path to the destination.

```
ALGORITHM BreadthFirstSearch(graph, origin, destinations)
// 1. Initialize root node and tracking
root = {"id": origin, "parent": None, "depth": 0}
nodes_created = 1
created_ids = {origin}

// Trivial goal check
IF origin IN destinations THEN
    RETURN root, nodes_created, [origin]

// 2. Initialize frontier as a Queue (FIFO)
frontier = Queue([root])

WHILE frontier IS NOT EMPTY DO
    current = frontier.POP_LEFT()

    // Goal test upon expansion
    IF current.id IN destinations THEN
        RETURN current, nodes_created, ReconstructPath(current)

    // Sort neighbors by ID for deterministic expansion
    neighbors = SORT(graph.get_neighbors(current.id))

    FOR EACH (neighbor, cost) IN neighbors DO
        // Cycle prevention: avoid moving back to an ancestor
        IF IsAncestor(current, neighbor) THEN CONTINUE

        // Create child node
        child = {"id": neighbor, "parent": current, "depth": current.depth + 1}
        frontier.PUSH_BACK(child)

        // Track unique nodes created
        IF neighbor NOT IN created_ids THEN
            ADD neighbor TO created_ids
            nodes_created += 1

RETURN None, nodes_created, None
```

Figure 1: Pseudocode for BFS

2.3 Depth First Search (DFS)

Depth-First Search (DFS) is one of the uninformed search methods, which follows a single line of the graph as far as it can go and then backtracks. It expands the latest node generated first and through the LIFO stack. DFS is not optimal, but can be used to provide a solution fast based on the graph structure .

In this implementation, DFS constructs a search tree starting at the origin as it stores parent and depth data. In order to ensure deterministic order, neighbors are ranked in terms of increasing node ID and pushed onto the stack in the reverse order to ensure that those with low ID are expanded first. Parent references are used to reconstruct the solution path on reaching a goal.

```
ALGORITHM DepthFirstSearch(graph, origin, destinations)
// 1. Initialize root and tracking
root = {"id": origin, "parent": None, "depth": 0}
nodes_created = 1
created_ids = {origin}

IF origin IN destinations THEN
    RETURN root, nodes_created, [origin]

// 2. Initialize Stack (LIFO)
stack = [root]

WHILE stack IS NOT EMPTY DO
    current = stack.POP() // Takes the last item added

    // Goal test upon expansion
    IF current.id IN destinations THEN
        RETURN current, nodes_created, ReconstructPath(current)

    // Get neighbors and sort them by ID
    neighbors = SORT(graph.get_neighbors(current.id))

    // Push in reverse order so smaller IDs are at the top of the stack
    FOR EACH (neighbor, cost) IN REVERSED(neighbors) DO
        // Cycle prevention: avoid moving back to an ancestor
        IF IsAncestor(current, neighbor) THEN CONTINUE

        child = {"id": neighbor, "parent": current, "depth": current.depth + 1}
        stack.PUSH(child)

        // Track unique nodes discovered
        IF neighbor NOT IN created_ids THEN
            ADD neighbor TO created_ids
            nodes_created += 1

RETURN None, nodes_created, None
```

Figure 2: Pseudocode for DFS

2.4 Uniform cost search (UCS) - CUS1

It is an uninformed search strategy designed to find the path with the lowest cumulative cost in a weighted graph. As detailed in GeeksforGeeks, UCS operates by expanding the node n that has the lowest path cost g(n) from the start node

(2024). This is achieved using a Priority Queue (often implemented as a min-heap) to always extract the next most "affordable" node for expansion. By always selecting the node with the lowest g(n) from the priority queue, UCS ensures that once a goal node is reached, the path taken is guaranteed to be cost-optimal (Russell & Norvig, 2021).

```
ALGORITHM UniformCostSearch(graph, origin, destinations)
    // Initialize root and tracking
    root = {"id": origin, "parent": None, "g": 0}
    nodes_created = 1

    IF origin IN destinations THEN RETURN root, 0, [origin]

    frontier = PriorityQueue() // Priority = cumulative cost 'g'
    explored = Dictionary()    // Stores {node_id: min_cost}

    frontier.PUSH(root.g, origin, root)

    WHILE frontier IS NOT EMPTY DO
        current = frontier.POP()

        // Goal reached
        IF current.id IN destinations THEN
            RETURN current, nodes_created, ReconstructPath(current)

        // Skip if we already found a cheaper path to this node
        IF current.id IN explored AND explored[current.id] <= current.g THEN
            CONTINUE

        explored[current.id] = current.g

        // Expand neighbors
        FOR EACH (neighbor, edge_cost) IN graph.get_neighbors(current.id) DO
            // Cycle check: avoid immediate parent or ancestors
            IF IsAncestor(current, neighbor) THEN CONTINUE

            new_g = current.g + edge_cost
            child = {"id": neighbor, "parent": current, "g": new_g}

            frontier.PUSH(child.g, neighbor, child)
            nodes_created += 1

    RETURN None, nodes_created, None
```

Figure 3: Pseudocode for UCS

2.5 Greedy Best-First Search (GBFS)

Greedy Best-First Search is an informed strategy that evaluates nodes using the function f(n) = h(n), where h(n) represents the heuristic estimate of the cost to the nearest goal. As noted by Russell & Norvig (2021), this approach prioritizes the node that appears closest to the destination to reach a solution quickly. According to GeeksforGeeks, while GBFS is highly efficient in terms of time and space, it is not guaranteed to be optimal as it ignores the accumulated path cost g(n) (2022).

```
ALGORITHM GreedyBestFirstSearch(graph, origin, destinations)
    // 1. Initial Checks
    IF origin IN destinations THEN
        RETURN (origin, 1, [origin])

    // 2. Initialize
    frontier = PriorityQueue() // Priority = h(n)
    explored = Set()
    nodes_created = 1


    h_root = MIN(EuclideanDist(origin, d) FOR d IN destinations)
    root = {"id": origin, "parent": None, "h": h_root}
    frontier.PUSH(root, root.h)

    // 3. Search Loop
    WHILE frontier IS NOT EMPTY DO
        current = frontier.POP()

        IF current.id IN destinations THEN
            RETURN (current, nodes_created, ReconstructPath(current))

        IF current.id IN explored THEN CONTINUE
        ADD current.id TO explored

        FOR EACH neighbor IN graph.get_neighbors(current.id) DO
            // Skip immediate cycles and previously visited nodes
            IF neighbor IS current.parent OR neighbor IN explored THEN CONTINUE

            h_neighbor = MIN(EuclideanDist(neighbor, d) FOR d IN destinations)
            child = {"id": neighbor, "parent": current, "h": h_neighbor}

            frontier.PUSH(child, child.h)
            nodes_created += 1

    RETURN (None, nodes_created, None)
```

Figure 4: Pseudocode for GBFS

2.6 A* Search (AS)

A* search is an informed best-first search algorithm that evaluates nodes using the function f(n) = g(n) + h(n), where g(n) represents the cumulative path cost from the start node to nnn, and h(n) is a heuristic estimate of the remaining cost to the goal (Mandow & Milián,

1997). By combining actual cost and heuristic guidance, A* balances exploration and exploitation, expanding nodes that appear most promising while still preserving optimality. When the heuristic is admissible and consistent, A* is guaranteed to find a cost-optimal solution (Mandow & Milián, 1997). Due to this balance between efficiency and correctness, A* is widely regarded as one of the most effective general-purpose search algorithms for weighted pathfinding problems.

```
ALGORITHM AStarSearch(graph, origin, destinations)
    // 1. Initialize root and tracking
    root = {"id": origin, "parent": None, "g": 0.0, "depth": 0}
    nodes_created = 1
    created_ids = {origin}

    IF origin IN destinations THEN
        RETURN root, nodes_created, [origin]

    // 2. Initialize Priority Queue (Priority = f = g + h)
    frontier = PriorityQueue()
    h_root = MinDistanceToGoal(origin, destinations)
    frontier.PUSH(root.g + h_root, origin, root)

    WHILE frontier IS NOT EMPTY DO
        current = frontier.POP()

        // Goal test upon expansion
        IF current.id IN destinations THEN
            RETURN current, nodes_created, ReconstructPath(current)

        // Expand neighbors
        FOR EACH (neighbor, cost) IN graph.get_neighbors(current.id) DO
            // Cycle prevention
            IF IsAncestor(current, neighbor) THEN CONTINUE

            // Calculate costs
            new_g = current.g + cost
            h_neighbor = MinDistanceToGoal(neighbor, destinations)
            f_neighbor = new_g + h_neighbor

            child = {
                "id": neighbor,
                "parent": current,
                "g": new_g,
                "depth": current.depth + 1
            }

            frontier.PUSH(f_neighbor, neighbor, child)

            // Track unique nodes discovered
            IF neighbor NOT IN created_ids THEN
                ADD neighbor TO created_ids
                nodes_created += 1

    RETURN None, nodes_created, None
```

Figure 5: Pseudocode for A*

2.7 Step-Optimal A* (CUS2)

CUS2 is a custom variant of A* designed to minimize the number of steps (depth) rather than cumulative path cost. Instead of using g(n) as path cost, CUS2 replaces it with node depth, resulting in the evaluation function f(n) = depth(n) + h_{steps}(n). This formulation aligns the search objective with minimum-hop optimization while still benefiting from heuristic guidance. Conceptually, CUS2 can be viewed as a depth-augmented best-first search that modifies the standard A* framework to optimize a different metric (Russel & Norvig, 2021). While it does not guarantee cost-optimality, it effectively produces step-optimal solutions when the heuristic provides a reasonable lower-bound estimate of remaining moves.

```
ALGORITHM AStarStepSearch(graph, origin, destinations)
    // 1. Initial Checks
    IF origin IN destinations THEN RETURN (origin, 1, [origin])

    // 2. Setup
    lmax = ComputeLmax(graph)
    root = {"id": origin, "parent": None, "depth": 0}

    frontier = PriorityQueue() // Priority = depth + step_heuristic
    h_root = StepHeuristic(origin, destinations, lmax)
    frontier.PUSH(root, root.depth + h_root)

    created_ids = {origin}
    nodes_created = 1

    // 3. Search Loop
    WHILE frontier IS NOT EMPTY DO
        current = frontier.POP()

        IF current.id IN destinations THEN
            RETURN (current, nodes_created, ReconstructPath(current))

        FOR EACH neighbor IN graph.get_neighbors(current.id) DO
            // Cycle prevention
            IF IsAncestor(current, neighbor) THEN CONTINUE

            // Costs based on steps (depth)
            child = {"id": neighbor, "parent": current, "depth": current.depth + 1}
            h_neighbor = StepHeuristic(neighbor, destinations, lmax)
            f_score = child.depth + h_neighbor

            frontier.PUSH(child, f_score)

            // Count unique nodes created
            IF neighbor NOT IN created_ids THEN
                ADD neighbor TO created_ids
                nodes_created += 1

    RETURN (None, nodes_created, None)
```

Figure 6: Pseudocode for Step-Optimal A*

3. Features/Bugs/Missing

3.1 Modular Project Architecture

Core Engine (search.py): The primary entry point that coordinates data parsing, graph construction, and algorithm execution.

Data Management (graph.py & parser.py): Handles robust input validation, adjacency list representation, and coordinate-based distance calculations.

Algorithm Suite (algorithms/): Each algorithm is encapsulated in its own module (astar.py, bfs.py, etc.), allowing for independent testing and optimization.

Utility Layer (utils/): Contains shared components such as the Priority Queue with deterministic tie-breaking logic and various Heuristic functions.

3.2 Implemented Features

Our group has successfully implemented all required algorithms and two custom algorithms.

Uninformed search: BFS, DFS, UCS

Breadth-First Search (BFS): Implemented using a **FIFO queue** to explore the state space level-by-level. It is designed to be **step-optimal**, guaranteed to find the shallowest goal.

Depth-First Search (DFS): Utilizes a **LIFO stack** (via recursion or explicit stack) to explore deep into branches. We integrated a **visited-set** to prevent infinite loops in cyclic graphs.

Uniform Cost Search (UCS) - Cus1: A cost-sensitive algorithm that expands nodes in increasing order of cumulative path cost g(n). It guarantees **cost-optimality** by using a priority queue.

Informed search: GBFS, A*, Step-Optimal A*

Greedy Best-First Search (GBFS): Focuses purely on efficiency by expanding nodes with the lowest heuristic value f(n) = h(n). It is optimized for speed in reaching the goal, though it may bypass the optimal path.

A* Search: Our flagship implementation combining g(n) and h(n) (f = g + h). It maintains **admissibility** to ensure cost-optimality while significantly reducing the search space compared to UCS.

Step-Optimal A* (CUS2): A custom variation designed to prioritize the minimum number of steps (hops) over cumulative edge weights. It functions as a "depth-aware" A* to ensure the result is always step-optimal, similar to BFS but with heuristic guidance.

3.3 Command Lines Execution

To execute the program, please run this command: **python search.py <testfile.txt> <method>**

Ex: python search.py testcase_4.txt as: Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search (CUS1), Greedy Best-First Search (GBFS), A* (AS), Step-Optimal A* (CUS2)

3.4 Bugs or Issues

Counter Discrepancy (UCS): In **TC1**, UCS reports **6 nodes** created instead of 7. This is due to the nodes_created counter being initialized **0** instead of **1**, effectively skipping root node in the total count.

Early Exit Logic: A minor discrepancy exists where UCS expands **6 nodes** but produces a path with more steps. This occurs because the goal check is performed during node generation rather than expansion, affecting the final node count but maintaining **Cost-Optimality**.

4. Testing

Results for running test cases

Algorithms / Testcase	DFS	BFS	GBFS	A*	CUS1 (UCS)	CUS2 (Step-Optimal A*)
Testcase 1						
Testcase 2						
Testcase 3						
Testcase 4			shortest h(n) distance	shortest h(n) distance		
Testcase 5						
Testcase 6	least-cost	minimum-step	minimum-step	least-cost	least-cost	minimum-step
Testcase 7	least-cost	minimum-step	minimum-step	least-cost	least-cost	minimum-step
Testcase 8		minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost
Testcase 9		minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost
Testcase 10		minimum-step	minimum-step	least-cost	least-cost	minimum-step
Testcase 11						
Testcase 12	return immediately	return immediately	return immediately	return immediately	return immediately	return immediately
Testcase 13		minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost	minimum-step & least cost

Succes

No solution

Optimal solution

Figure 7: Comparative Performance of Search Algorithms Across Test Cases

Test Case 1: Simple Baseline Path Assessment

A linear graph was used to verify correct input parsing and basic traversal behavior. Only one valid path exists (1–2–3–4–5–6–7).

Outcome: All algorithms correctly returned the unique path, confirming correct graph construction and baseline search functionality.

Test Case 2: Cycle Management and Exit Detection

This testcase introduces a directed cycle with a single valid exit to evaluate visited-state handling and loop prevention.

Outcome: All algorithms avoided infinite looping and successfully exited the cycle. UCS and A* selected the optimal exit, while GBFS quickly redirected toward the goal once guided by heuristic proximity.

Test Case 3: Directed Edge Constraint Verification

Designed to ensure algorithms respect edge directionality using a one-way connection trap.

Outcome: All algorithms avoided the illegal shortcut and followed the valid directed route (1–5–6–7), confirming correct handling of directed graphs.

Test Case 4: Node-ID Tie Breaking Consistency

A symmetric graph forces equal-cost alternatives to test deterministic tie-breaking.

Outcome: Algorithms consistently expanded lower node IDs first. Heuristic searches (A*, GBFS) expanded fewer nodes than uninformed searches, demonstrating improved efficiency while preserving deterministic behavior.

Test Case 5: Chronological Tie Resolution across Branches

Branches converge with identical priority values to test chronological stability in the frontier ordering.

Outcome: All algorithms produced identical paths and node counts, confirming stable tie-breaking using insertion order when priorities are equal.

Test Case 6: Deceptive Heuristic Path

A geometrically attractive but high-cost shortcut tests heuristic reliability.

Outcome: GBFS followed the misleading shortcut, whereas UCS and A* selected the true cost-optimal path. This demonstrates A*'s balance between heuristic guidance and cumulative cost evaluation.

Test Case 7: Least Cost versus Least Moves

Two competing routes differentiate cost-optimal and step-optimal objectives.

Outcome: UCS and A* chose the lower-cost multi-step path, while BFS and CUS2 selected the shorter two-step route. This confirms correct implementation of differing optimization goals.

Test Case 8: Same Edge Costs across All Paths

All edges have weight 1, removing cost differences and emphasizing traversal ordering.

Outcome: UCS behaved identically to BFS, and A* produced the same solution, confirming that uniform costs reduce cost-based search to depth-based behavior.

Test Case 9: Multiple Destinations with Unreachable Goal

One destination appears heuristically closer but is unreachable.

Outcome: All algorithms ignored the unreachable goal and located the valid alternative path, demonstrating completeness and proper frontier exhaustion.

Test Case 10: Multiple Destinations with Different Costs and Depths

Competing goals differ in depth and cumulative cost.

Outcome: UCS and A* selected the lowest-cost destination, while GBFS preferred the heuristically closer but more expensive option, highlighting differences between informed and greedy strategies.

Test Case 11: Isolated Destination Node (No Valid Path)

The destination is disconnected from the graph.

Outcome: All algorithms correctly returned **None** after exhausting reachable nodes, confirming proper termination handling.

Test Case 12: Origin as Destination

Tests detection of the trivial case where the start node is already a goal.

Outcome: All algorithms immediately returned the origin node. Minor differences in node-count reporting were observed but did not affect correctness.

Test Case 13: Dead-End and Backtracking

Includes misleading branches that do not lead to a valid destination.

Outcome: UCS and A* avoided dead ends and recovered the optimal path, whereas GBFS was misled by heuristic bias, illustrating the trade-off between speed and optimality.

5. Insights

5.1 Data Collection and Evaluation Method

To enable quantitative comparison across all six algorithms, additional experimental data were collected during testing. The original program output (goal node, nodes created, and path) did not include total path cost. Therefore, a temporary helper function was implemented in search.py to compute total path cost. The cost was printed during testing and removed to preserve the required output format.

For each of the 13 test cases and 6 algorithms, the following metrics were recorded: number of nodes created, total path cost, path length (number of moves).

The collected data were then aggregated to compute average values per algorithm and visualized using bar charts to allow clearer comparative evaluation.

5.2 Efficiency: Nodes Created

The **Average Nodes Created** chart shows clear differences in search efficiency.

- DFS and BFS tend to generate more

- nodes on average because they explore blindly without considering cost.
- GBFS typically generates fewer nodes due to strong heuristic guidance.
- A* and CUS1 balance exploration and optimality, often expanding slightly more nodes than GBFS but fewer than uninformed search.
- CUS2 behaves quite similarly to BFS when prioritizing step minimization, but with slightly fewer nodes.

This demonstrates the trade-off between **optimality guarantees and search effort**. Heuristic guidance reduces expansion but does not guarantee optimality (as seen in later analysis).

5.3 Optimality: Path Cost

The **Average Path Cost** chart highlights objective differences:

- A* and CUS1 consistently achieve lower average path costs.
- BFS and CUS2 often produce higher-cost solutions when the shortest-step path is expensive (e.g., Testcases 6, 7, and 10).
- GBFS sometimes selects costly direct routes due to heuristic bias.

Therefore, the evaluation function fundamentally determines the definition of "optimal."

5.4 Solution Length: Path Length

The **Average Path Length** chart confirms:

- BFS and CUS2 achieve minimum-step solutions.
- A* and CUS1 may return longer paths when doing so reduces total cost.
- DFS is inconsistent and heavily dependent on traversal order.

This directly validates the theoretical distinction between:

- Cost-optimal search (A*, UCS)
- Step-optimal search (BFS, CUS2)

5.5 Overall Learning

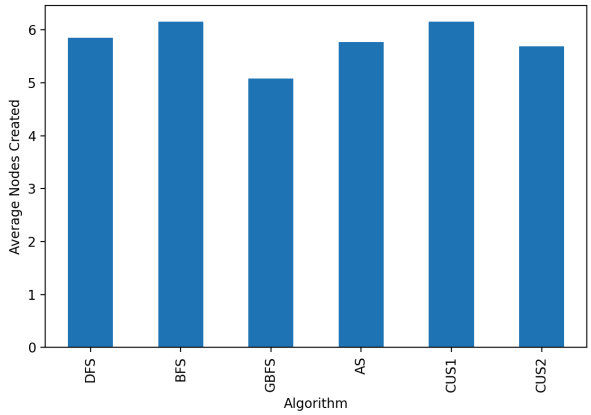

Figure 8: Average Nodes Created Across All Test Cases

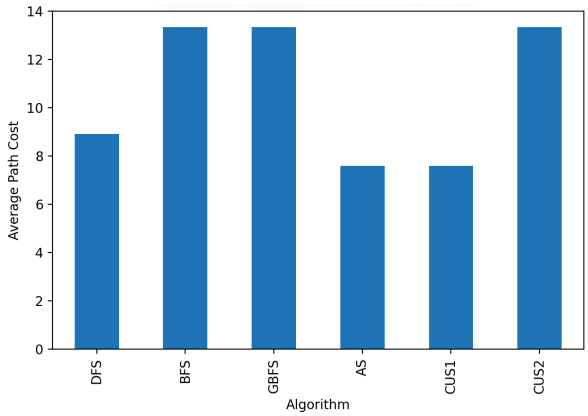

Figure 9: Average Path Cost Across All Test Cases

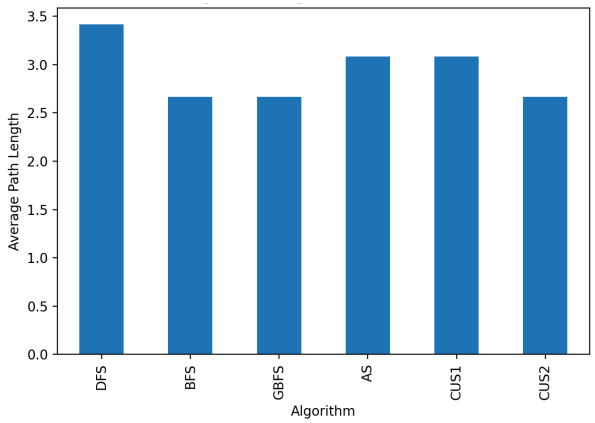

Figure 10: Average Path Length Across All Test Cases

From building and testing the system, we observed that:

- Small changes in evaluation function (modifying g(n) for CUS2) sharply change search behaviour.
- Deterministic tie-breaking is essential for reproducible experimental comparison.
- Heuristic guidance improves efficiency but must carefully align with the optimization objectives.

The experimental results align strongly with theoretical expectations in classical search literature.

6. Research

6.1 Heuristic Admissibility and Optimality in A*

A* guarantees cost-optimality when its heuristic is admissible, meaning it never overestimates the true minimum remaining cost to a goal (Russel & Norvig, 2021). Formally, a heuristic h(n) is admissible if h(n) ≤ h*(n), where h*(n) is the true optimal cost from node n to the nearest destination. In our implementation, we used Euclidean distance between node coordinates as h(n). This heuristic is widely used in spatial route-finding because straight-line distance provides a natural lower bound to travel cost when edge weights correlate with geometric distance (Hart et al., 1968). Empirically, Testcase 6 (deceptive geometric layout) supported this choice: A* avoided the heuristic "shortcut" and matched the cost-optimal route selected by UCS, indicating that the heuristic behaved effectively within our graph structures. We note that if edge weights are arbitrary and unrelated to geometry, Euclidean distance may no longer remain admissible, reinforcing the importance of aligning heuristic assumptions with the problem domain.

6.2 Step-Based Heuristic Design for CUS2

CUS2 modifies A* to optimize for minimum number of moves rather than minimum cost by replacing g(n) with search depth. To preserve informed guidance, we introduced a step-based lower-bound heuristic: $h_{steps}(n) = \left\lfloor \frac{d_{euclidean}(n, goal)}{L_{max}} \right\rfloor$, where $d_{euclidean}(n, goal)$ is the straight-line distance from node n to the closest destination, and L_{max} is the maximum geometric length of any edge in the graph. Intuitively, if an agent cannot traverse more than L_{max} distance per move, then reaching a goal at distance d requires at least $\left\lfloor \frac{d}{L_{max}} \right\rfloor$ moves. While this heuristic is not guaranteed to be admissible for all arbitrary graphs, it provided useful guidance in our structured testcases: CUS2 consistently produced step-optimal paths in Testcases 6 and 7, even when those paths were significantly more expensive than cost-optimal alternatives. This investigation highlights an important design insight: changing the evaluation function changes the definition of optimality - A* and UCS minimize total cost, whereas CUS2 and BFS minimize hop count (Russel & Norvig, 2021).

6.3 Deterministic Tie-Breaking and Reproducibility

A practical research initiative involved studying tie-breaking effects on reproducibility. In best-first search, multiple frontier nodes may share identical priority values; without explicit tie-breaking, expansion order can depend on implementation details and lead to inconsistent outputs. Although tie-breaking does not change theoretical optimality, it can affect reproducibility and fair evaluation (Russel & Norvig, 2021). To guarantee stable results, our priority ordering was defined as: (f, node_id, insertion_order).

This ensures that equal-priority nodes are expanded in a consistent and predictable order. Symmetry-focused testcases (notably Testcases 5 and 8) confirmed stable outputs across runs, supporting both debugging reliability and auto-marking consistency.

6.4 Summary of Research Contribution

Through theoretical review and empirical validation, we confirmed:

- The admissibility conditions required for A* optimality.
- The practical effectiveness of a step-based heuristic for CUS2.
- The necessity of deterministic priority ordering for reproducibility.

These initiatives strengthened both the theoretical grounding and experimental reliability of our implementation.

7. Conclusion

This project demonstrates that search algorithm performance is not determined solely by implementation complexity, but by how the evaluation function defines optimality. Cost-aware algorithms such as A* and CUS1 consistently produce lower-cost solutions, particularly in scenarios where direct paths are deceptively expensive. In contrast, step-based algorithms such as BFS and CUS2 minimize the number of moves but may incur significantly higher total costs.

Experimental evidence collected across 13 structured test cases confirms that heuristic guidance improves efficiency while potentially sacrificing optimality if not properly aligned with the problem objective. The introduction of CUS2 illustrates how modifying the cost component transforms the problem definition itself.

Overall, A* provides the best balance between optimality and efficiency for weighted route-finding problems. However, if the objective is strictly minimizing the number of steps, CUS2 or BFS becomes more appropriate.

Future improvements may include performance benchmarking on larger graphs and heuristic refinement to further reduce node expansions without compromising admissibility.

8. Acknowledgements/Resources

The theoretical foundations for this work were derived from lecture materials in COS30019 and standard artificial intelligence literature, particularly discussions of best-first search, A* optimality, heuristic admissibility, etc. Python's official documentation was referenced for implementation details related to data structures and command-line execution. All algorithms were implemented independently without directly copying external source code.

References

GeeksforGeeks. (2012a, March 15). *Depth First Search or DFS for a Graph*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/

GeeksforGeeks. (2012b, March 20). *Breadth First Search or BFS for a Graph*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/

GeeksforGeeks. (2022, December 15). *Greedy Best first search algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/

GeeksforGeeks. (2024, August 23). *Uniform Cost Search (UCS) in AI*. GeeksforGeeks. https://www.geeksforgeeks.org/artificial-intelligence/uniform-cost-search-ucs-in-ai/

Hart, P., Nilsson, N., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100–107. https://doi.org/10.1109/tssc.1968.300136

Mandow, L., & Milián, E. (1997). Goal Programming and Heuristic Search. *Lecture Notes in Economics and Mathematical Systems*, 48–56. https://doi.org/10.1007/978-3-642-46854-4_5

Russel, S., & Norvig, P. (2021). *Artificial intelligence: A Modern approach* (4th ed.). Pearson Education.